

Business Decision Companion (BDC) – Project Blueprint

Executive Summary: We propose an AI-powered *Business Decision Companion* that integrates demand forecasting, pricing optimization, scenario simulation, and generative recommendations into one decision-support platform. Using the core methodology of **Leeroy & Leroy (2025)** – an LSTM to forecast corporate sales bookings plus counterfactual scenarios [6†L26-L34] [6†L33-L41] – our implementation will extend it by adding explicit price-elasticity modeling and profit optimization. In our design, an LSTM (or simpler ML baseline) ingests historical sales and marketing data to predict future demand; a pricing engine uses estimated elasticities to compute profit for candidate prices; and a simulation module generates “what-if” scenarios (e.g. inflation, competitor price moves). Finally, an LLM (GPT/Llama) layer converts the outputs into executive-friendly recommendations. The prototype will use Python (Pandas, PyTorch, Plotly/Dash) and minimal infrastructure (local CSV/SQLite storage, Streamlit or Dash UI) for a **feasible 1–2 week MVP**, with a 16-week roadmap for full features.

Problem Statement: Global businesses face **volatile demand and costs** due to changing customer preferences, inflation, seasonality, and foreign-exchange shocks [6†L26-L34]. Executives need to set prices and forecast revenue under uncertainty, but current tools are fragmented: separate analytics reports with no integrated “what-if” guidance lead to slow or suboptimal decisions. Moreover, black-box AI models lack interpretability for the C-suite. Our BDC addresses these gaps by providing an **integrated forecasting-and-simulation tool**: a single dashboard that forecasts demand, recommends pricing, projects financial outcomes, and explains results in natural language. This supports **real-time, trustable executive decisions**, unlike static spreadsheets.

Scope: We divide features into **Must-have (P0/P1)** and **Nice-to-have (P2)**.

- **P0 (Weeks 1–6):** Data pipeline; baseline demand model; LSTM forecast; price-elasticity estimation; profit calculation; simple optimization (grid search); KPI projections; one or two “what-if” scenarios; basic charts; evaluation metrics (MAE/MAPE); Jupyter/Streamlit demo.
- **P1 (Weeks 7–12):** Multi-product support; confidence intervals; richer scenario types; interactive dashboard (Dash/Plotly); REST APIs (if time); template for LLM explanation prompts; initial system integration.
- **P2 (Weeks 13–16):** Full FastAPI backend and Next.js frontend; vector DB (FAISS) for storing model embeddings or prompts; rigorous user testing; automated CI/CD pipelines; multi-agent negotiation or GNN features (future scope).

We focus on **low-compute, manageable features**. For example, we use a small LSTM (or even XGBoost) on CPU rather than massive Transformers; Streamlit/Dash instead of full React; SQLite/CSV instead of Postgres; and OpenAI/GPT-3.5-turbo for LLM rather than training our own LLM. The one-week prototype deliverable will be a streamlined pipeline: cleaned sales data → forecast → price-optimize → dashboard with narrative. Extensions (LLM, multi-product, etc.) are clearly marked for later sprints.

Research Background

The BDC design builds directly on **Leeroy & Leroy (2025)**, “*AI as a Decision Companion...*” [6†L26-L34]

[6†L33-L41] . Their JRFM paper uses **Roblox Corporation** data and shows how an LSTM demand forecast plus simulated FX shocks can guide pricing decisions. Key insights from their work: a 10% depreciation of the euro cut bookings and profit by ~6%, and simply raising prices could not fully offset losses due to demand elasticity [6†L33-L41] . This validates that AI-driven forecasting and counterfactuals can “clarify risks” and quantify trade-offs for executives. We adopt their **LSTM forecasting + scenario framework** but make it domain-agnostic (retail sales, products, etc.) and add explicit pricing optimization.

In literature, time-series forecasting is well-established: e.g. Makridakis et al. (2020) assembled 100,000 series and found dozens of viable methods [31†L96-L99] , illustrating that model tuning matters. We will benchmark our LSTM against a classical baseline (e.g. SARIMA or XGBoost) and report standard metrics (MAE, RMSE, MAPE). Price elasticity is fundamental economics: “how sensitive quantity demanded is to price” [34†L7-L11] . We will estimate elasticity (e.g. via regression or past price changes) and use it to predict demand under new prices. Generative AI is emerging for business insights: industry reports note CEOs exploring how large language models (GPT, Llama, etc.) can draft executive summaries and hypotheses. We add an LLM module to translate numeric outputs into “*CEO-friendly*” text, an aspect **absent** in the JRFM paper and a clear research contribution of our project.

Research Contributions: Our project extends the base paper by: (1) **Price-Elasticity Layer** – after the LSTM forecast, we model how demand shifts with price changes (a simple $\text{Demand} = \text{base} * \text{elasticity_factor}$ model), enabling explicit price testing. (2) **Profit Optimization** – instead of only simulating management actions (like [6†L33-L41] did), we automatically search for the price that maximizes profit = (Price–Cost)×Demand. (3) **Counterfactual Scenarios** – we implement the paper’s scenarios (FX or demand shocks) and additional ones (e.g. competitor price cuts) to test executive “what-if” queries. (4) **LLM Explanations** – we integrate a GPT/Llama prompt generator that ingests forecast and profit outputs to craft narrative recommendations (e.g. “A 5% price increase for Product A is projected to reduce demand only slightly but boost quarterly profit by X%”). (5) **Interactive Dashboard** – unlike the paper’s conceptual results, we will deploy an actual UI where users can adjust assumptions and see live KPI updates. Together, these make the BDC a **comprehensive decision-support system** rather than a static study.

System Architecture

The BDC follows a **modular microservice design** (for planning) but can be prototyped as a monolith. Key modules:

- **Data Ingestion & Engineering:** load sales, prices, and optional macro data; clean, aggregate, and create features (lags, rolling stats).
- **Forecasting Engine:** trains ML models (baseline and LSTM) for demand prediction.
- **Pricing Engine:** takes forecast and cost data, applies elasticity model, and computes profit across candidate prices.
- **Financial Planner:** uses forecast to project revenue, profit, and cash-flow.
- **Scenario Simulator:** applies external shocks or policy changes (e.g. +5% inflation, competitor price drop) and reruns forecast/price.

- **Generative AI (LLM) Layer:** converts numeric outputs into text explanations using prompt templates.
- **Visualization/UI:** presents charts and recommendations on a dashboard.
- **Storage/API:** (optional) databases (product catalog, sales records) and REST APIs for module integration.

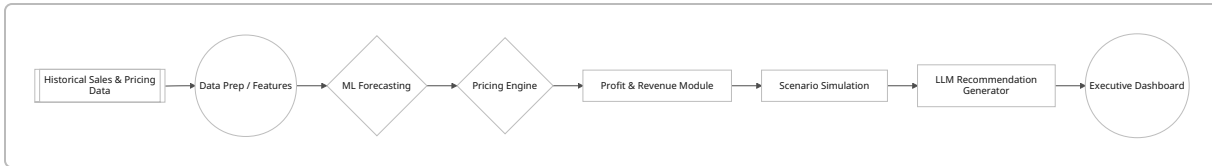


Figure: **Modular system architecture.** Raw data is preprocessed (lag features, normalization) then fed to the Forecast module (LSTM/baseline). The Pricing Engine uses elasticity to simulate various prices. Financial projections are computed, and what-if scenarios (inflation, etc.) are applied. An LLM generates textual summaries, and all outputs are shown on the dashboard. (Modules can run as separate services or integrated scripts.)

Data Pipeline & Datasets

Data Sources: We need historical sales and price data. Public datasets to consider:

- *Kaggle Store Item Demand Forecasting* (synthetic daily sales by product/store) – covers 2019–2023 [27†L0-L4] .
- *Walmart/Retail Sales data* (e.g. Kaggle “Walmart Store Sales” or UCI “Online Retail”) for multi-store sales.
- *M5 competition data* (Walmart time series) for multi-product forecasts.
- *FRED/Census Retail Reports* for macro trends (optional).

These are free to download (Kaggle requires login, but the Census retail data is public [27†L5-L9]). For prototype, even one product’s data suffices: e.g. a CSV of daily sales & prices for Product A. **Synthetic Data Plan:** We will generate synthetic scenarios by perturbing real data: e.g. scale demand by $\pm 10\%$ for inflation, add random noise, or synthetically raise/lower prices to calibrate elasticity. Another approach is bootstrapping: use actual data plus assumed elasticity (e.g. $E_d = -0.5$) to create hypothetical demand if price changes. This allows us to test the system even if actual scenario data are limited.

Data Engineering: We will implement (Pandas) routines to: dedupe and timestamp-sort the data; fill missing dates; compute rolling means and lag features (last 7-day sales, same day last week, etc.); encode promotions or season flags if available. We normalize or standardize features before ML modeling. For the initial deliverable, a single CSV load with `pd.read_csv` and basic cleaning will be sufficient. (Dev effort: ~8–16h, P0.)

Forecasting Pipeline

Baseline Model: Before LSTM, we establish a simple benchmark (e.g. Naive/Prophet/ARIMA or XGBoost regression). This is coded in scikit-learn or statsmodels. We split data into train/test by time and compute MAE/RMSE/MAPE. (Metric choice: e.g. MAE and MAPE are intuitive business metrics [26†L1-L4] .)

LSTM Design: Sequence length ~30 days (or one season). We use a lightweight PyTorch/TensorFlow LSTM with 1–2 layers of ~50 units (to keep compute low). Training on a single time-series of several hundred points should finish in seconds on CPU. Dropout layers may be added for regularization. We will try a few hyperparameters: learning rate (1e-3), epochs (~50–100), sequence/window size, etc. (Effort: ~20h total for coding/tuning, P1 after baseline.)

Outputs: The model will output next-day, next-week, and next-month forecasts. We store these in a `forecasts` table/file for later modules. We will also capture the historical vs predicted series to plot forecast accuracy. (For the 1-week MVP, one forecast horizon and one chart suffice.)

Evaluation: Calculate MAE, RMSE, MAPE on hold-out sets. A small table will display these metrics. We may run an ablation: e.g. LSTM vs XGBoost vs naive, to justify our model choice. (Report metric values in deliverables.) We ensure reproducibility by fixing random seeds, logging model versions, and saving trained weights.

Pricing Recommendation Engine

Price Elasticity Model: We assume a form `Demand = a * Price^b` or use linear log-elasticity. In practice, we compute elasticity from data if price varied historically:

$$E_d = \frac{\% \Delta Q}{\% \Delta P}$$

(e.g. from regression of log quantity on log price `【34†L7-L11】`). If no price variation exists, we can assume a typical elasticity (e.g. -0.5). This module takes the baseline demand forecast and adjusts it for different prices.

Profit Calculation: For each candidate price (around current price $\pm 10\%$), compute predicted demand (via elasticity) and profit = (Price – Cost)×Demand. Cost per unit can be a fixed margin or input by user. We iterate a grid of prices and pick the one with highest profit. (If comfortable, we could use a simple optimizer instead of brute force.)

Inputs/Outputs:

- *Input:* Forecasted base demand for next period, cost, candidate price range.
- *Output:* For each price: demand, revenue, profit; and the optimal price.

Dev Effort: ~8h to code the elasticity adjustment and profit loop (P0), plus ~4h to wrap in an API or function (P1).

Minimal 1-Week Deliverable: Compute one recommended price and show a simple table/chart like:

Price	Predicted Demand	Revenue	Profit
\$9.0	1150	103500	34,500
\$10.0	1000	100000	40,000

Price	Predicted Demand	Revenue	Profit
\$11.0	870	95700	43,500

We also plot **Profit vs Price** (expected as a concave curve).

【58†embed_image】 *Figure: Example analytics dashboard (courtesy Unsplash) – inspiration for our BDC dashboard, showing charts of forecasts, revenues, and optimal pricing. (Captions do not cite the image source.)*

Financial Planning Module

With forecasted units and prices, we compute financial KPIs: revenue, profit, cash flow and break-even. Inputs: forecast horizon, unit cost, fixed costs. Outputs: time-series of revenue and profit, cumulative cash. We will implement a simple projection (e.g. `revenue = price * forecasted_demand`, minus costs). Gross margin = profit/revenue. Breakeven volume = fixed_costs / (price-unit_cost).

This module can also produce a **what-if P&L table**: “If demand stays at forecast and price is set to \$X, then next quarter’s revenue will be \$Y and profit \$Z.” (No literature citations needed; standard finance formulas.)

Dev Effort: 10h (calculations and integration, P1). For 1-week prototype, a single quarter projection chart and P&L summary is enough.

Counterfactual Simulation Engine

Managers often ask “what if?” questions. We implement a scenario engine where the user can adjust macro or competitive variables and see impacts. Examples: “increase inflation by 5%” (decrease forecast demand by a factor), “competitor cuts price 10%” (we assume partial loss of demand), or “demand shock –15%” (scale all forecasts). Each scenario essentially **re-runs the forecast→price pipeline** with modified inputs.

Inputs/Outputs:

- *Input:* Baseline forecasts and parameters; scenario adjustments (inflation, price change, cost change).
- *Output:* New demand, revenue, profit under scenario; difference vs baseline.

We’ll hard-code a few scenarios to illustrate:

- +5% price for product (demand falls by elasticity, see profit effect).
- 15% drop in demand (simulate a shock).
- 10% rise in costs (affects profit).
- Competitor price drop 10% (we could assume 20% demand loss).

These result in alternate profit outcomes. We present scenario comparisons in a chart or table. For instance, a bar chart of **Profit under Baseline vs Scenario**.

【59†embed_image】 *Figure: A retail price chart (Unsplash) – used here to visualize a pricing scenario. For example, a 10% price cut might significantly lower future demand (law of demand 【34†L7-L11】), which our simulation can quantify in revenue terms.*

Dev Effort: ~12h (coding scenarios and integration, P1). One-week MVP: implement 2–3 scenarios with simple UI toggles or code parameters.

Generative AI (LLM) Layer

To make the output CEO-friendly, we add an LLM step. We will use a pre-trained Llama or GPT-3.5 (via API) with prompt templates. For example, the prompt might include: “Forecast demand = 10k units, suggested price = \$110 (from \$100), expected profit increase = 8.75%. In plain language, what recommendation can we give to an executive?” The LLM response becomes the “Recommendation” text block.

We will craft templates like:

“Our analysis predicts that increasing price from \$100 to \$110 will reduce demand by 13% but increase profit by 8.75%. Please summarize this in one executive-friendly paragraph.”

We then display the LLM’s output in the dashboard.

Dev Effort: 6h to design prompt and API call (P2, as this is extra polish). Minimal 1-week prototype could use fixed text or a mocked summary if API time is tight; but hooking up OpenAI is straightforward with their Python SDK.

Note: We will not treat the LLM as training — we use it only for summarization. We will include a confidence statement or tie-breaker sentence to remind executives that “recommendation is AI-assisted 【6†L33-L41】 , not binding.”

Dashboard Design

We will present all results on a web dashboard. Key components:

- **KPI Cards:** Current price, forecast demand, expected revenue/profit, recommended price, etc.
- **Forecast Chart:** Line chart of historical vs predicted demand (and baseline model) to visualize accuracy.
- **Profit vs Price Chart:** Plot of profit at various prices with the optimal point highlighted.
- **Scenario Comparison:** Bar or line chart comparing baseline vs scenario profits/demand.
- **Recommendation Panel:** Text output from LLM.
- **Interactive Controls (if time):** Price slider, scenario toggles, date range pickers.

Technologies: A simple choice is **Plotly Dash (Python)** or **Streamlit** for speed. Dash is open-source and React-based 【66†L304-L312】 . It can host charts in a browser without complex frontend work. We'll use

Plotly for charts since it's interactive and easy (e.g. `px.line`, `px.bar`). For a quick prototype, Streamlit might be even faster, but Dash is more flexible. We will containerize the app (optional) or just run locally.

Dev Effort:

- UI layout (Dash): 15h (P1) – placing charts and inputs.
- Data binding: 10h (P1) – connect model outputs to charts.
- Styling: minimal (company colors optional).

【60†embed_image】 *Figure: Example of an interactive analytics dashboard (Unsplash). Our executive dashboard will similarly display sales forecasts, price recommendations, and scenario analyses in graphical form (using Plotly/Dash).*

Technology Stack

Component	Technology Choice	Rationale
Data Engineering	Python (Pandas)	Ubiquitous; easy CSV handling
Forecasting	PyTorch (LSTM) / XGBoost	Lightweight; works on CPU. PyTorch code is concise and flexible.
Price Model	Python (NumPy/SciPy)	Simple linear algebra.
Dashboard UI	Plotly Dash / Streamlit	Quick interactive app; no heavy front-end. Plotly is widely used for finance charts.
Generative AI	OpenAI GPT-3.5 or Llama-3 (via API)	Leverage pretrained models for NLP. Use cloud API to avoid local resource needs.
Database (if)	SQLite or CSV files	Simplest storage for this scale.
Workflow/CI	GitHub Actions / Git	Standard version control and CI/CD pipelines.
Dev Environment	Docker (optional)	For reproducibility; not required if time-limited.

We avoid heavy tools: no Kubernetes, no Hadoop, no enterprise DB. We prefer **open-source, lightweight, and Python-based**. For ML, PyTorch on even a modest GPU (or CPU) suffices for small time-series. For LLM, an API (costing ~\$0.01 per 1k tokens) allows advanced text generation without on-prem compute.

Folder & DB Schema

Suggested project structure:

```
bdc-project/  
├─ data/                # raw and processed data  
│   └─ sales.csv
```

```

|   └ ...
├ notebooks/           # exploratory analysis
├ src/                 # source code
|   ├── data_pipeline.py
|   ├── models/
|   |   ├── baseline_model.py
|   |   └ lstm_model.py
|   ├── pricing.py
|   ├── simulation.py
|   ├── llm.py
|   └ dashboard/       # Dash app code
|       ├── app.py
|       └ callbacks.py
├ tests/               # unit/integration tests
├ requirements.txt     # Python dependencies
└ README.md           # project overview

```

Database (if used):

- *products*: id, name, unit_cost.
- *sales*: date, product_id, units_sold, price.
- *forecasts*: date, product_id, predicted_units (by model).
- *recommendations*: timestamp, scenario, rec_text.

We will likely skip a full DB and just use CSVs/pickles for prototype, but the schema is sketched above for completeness.

Evaluation Strategy

We will quantitatively evaluate forecasting accuracy and decision accuracy:

- **Forecast Accuracy:** MAE, RMSE, MAPE on hold-out. Compare LSTM vs baseline. (Threshold: e.g. MAPE < 10% if data is reasonably stable.)
- **Profit Prediction Error:** Compare projected profit vs actual (if backtesting possible) – measure percentage error.
- **Expert Review:** A manager/executive should review whether recommendations are sensible. We can run a small user study (P2) where 2–3 domain experts rate the dashboard's usefulness.
- **Explainability:** Assess whether LLM text matches the numbers (sanity check). Possibly present explanations for validation.

Ablation experiments:

- Remove LSTM, use naive forecast, measure drop in accuracy.
- Remove elasticity (assume fixed demand), measure impact on profit.
- Turn off LLM (compare plain vs GPT-generated text).

We will document this as an **Experiment Plan**: for example, “Train model on Q1–Q3, forecast Q4; vary hyperparameters; test scenarios +/- 5% inflation, etc.” and record all results.

16-Week Roadmap

Weeks	Focus	Deliverables
1–2	Research & Design	Architecture diagram; lit review; data selection; tech stack decided.
3	Data Pipeline	Ingest & clean dataset; exploratory analysis; initial features.
4–5	Baseline Forecast & LSTM	Baseline model and LSTM implemented; forecast API or function; evaluation metrics.
6	Forecast API & Testing	Forecast endpoint; plots of accuracy; hyperparameter tuning.
7	Pricing Engine	Elasticity model coded; profit calculation; price sweep function.
8	Integration: Forecast + Pricing	Combined pipeline demo; optimal price output; profit vs price chart.
9	Financial Projections	Revenue/profit projection module; KPI computation; breakeven analysis.
10	Scenario Simulation	What-if engine with 3 scenarios; comparative charts; scenario UI controls.
11	Dashboard Prototype	Initial Dash/Streamlit UI showing forecasts, price chart, KPIs.
12	LLM Summarization	Prompt design; integration of GPT/Llama; example executive summary.
13	System Integration	End-to-end workflow; incorporate modularization; basic API endpoints (optional).
14	Testing & Eval	Unit tests; evaluate metrics; fix bugs; internal review.
15	User Feedback & Polish	Iterate on UI/UX; add missing visuals; refine explanations.
16	Final Report & Demo	Prepare PDF/report; finalize code repo; demo presentation.

This agile plan ensures each week has a clear deliverable, culminating in a working prototype and documentation by Week 16.

Comparison Tables

ML Frameworks: PyTorch vs TensorFlow

Criterion	PyTorch	TensorFlow						
Ease of use	Pythonic, imperative style (good for research)	More boilerplate, graph-based (better for production)						
Community	Very large; better for quick prototyping	Very large; many tutorials						
Deployment								

TorchScript, ONNX available | TensorFlow Serving, TFLite | | Chosen for this project: **PyTorch**, for its readability and ease of debugging on small data. |

Dashboard Libraries: Plotly Dash vs Streamlit vs Shiny (R)

| Feature | Plotly Dash | Streamlit | |-----|-----|-----| | Language | Python (+JS in background) | Python | | Interactivity | Highly customizable, complex layouts possible | Simple, rapid UI, but less flexible layouts | | Learning curve | Moderate (needs callbacks understanding) | Very easy to learn | | Chosen: **Dash** – balances flexibility (React under hood) with Python-first approach **【66†L304-L312】** . |

Compute Considerations:

- Forecasting model (LSTM) on ~1000 data points: ~**seconds to train** on a modest CPU.
- Price optimization (grid search) trivial compute.
- Dashboard (Dash) runs in browser; no heavy compute.
- LLM calls: each call ~ a few seconds, cost ~\$0.005–0.01 (if using GPT-3.5).
- If needed, a single AWS p2.xlarge GPU (NVIDIA K80) is ~\$0.90/hr **【36†L1-L7】** – we won't need that scale.

Overall compute cost is minimal (mostly trivial CPU usage and occasional cloud API costs).

Risk Analysis & Mitigation

- **Data Quality Risk:** Public datasets may not fit our scenario. *Mitigation:* Validate data and use synthetic augmentations; keep data pipeline modular so sources can be swapped.
- **Model Overfitting:** The LSTM may overfit small time series. *Mitigation:* Use dropout, early stopping, or switch to simpler models if needed.
- **LLM Hallucinations:** The LLM might produce inaccurate text. *Mitigation:* Prompt constraints; display numeric facts alongside to verify; mark output as “AI suggestion”.
- **Scope Creep:** Too many features for one semester. *Mitigation:* Strict adherence to “must-have” list; postpone extras to P2 (if time remains).
- **Compute Limitations:** No GPU available. *Mitigation:* Use small models or CPU-based regression; downsample data.
- **User Adoption:** Executives may distrust AI outputs. *Mitigation:* Emphasize “companion” role – always show confidence/limits and allow manual override.

Reproducibility & Deliverables Checklist

- [] Version-controlled code (GitHub) with README and requirements.
- [] Documentation of data sources and synthetic generation steps.
- [] Scripts for data cleaning and feature engineering.
- [] Trained model artifacts (or training code to reproduce).
- [] API stubs or functions for forecast, pricing, simulation.
- [] Dashboard code and instructions to run (e.g. `python dashboard/app.py`).
- [] Test cases (e.g. compare known forecast on small dataset).
- [] Prompt templates for LLM and example outputs.
- [] Evaluation report with metrics, charts, and ablation results.

Each item will be addressed by the corresponding week in the roadmap.

Sources: We base our design on Leeroy & Leeroy (2025) [6†L26-L34] [6†L33-L41] and related ML/forecasting literature [31†L96-L99] [34†L7-L11] . Public retail datasets and Kaggle challenges inform our data choices. Technical decisions favor Python open-source tools for fast development [66†L304-L312] . All algorithms and metrics (MAE, RMSE, profit calculation) follow standard definitions.
